

Software Development Fundamentals (SDF) – I

ODD 2022



File Handling

**Jaypee Institute of Information Technology (JIIT)
, Noida**

Outline:

Reading and Writing a Binary File in C
Special streams
Command line Arguments

Reading and Writing a Binary File in C

- We use the `fread()` and `fwrite()` function to read and write data from a binary file respectively.
- Working with binary files is a bit more complicated than working with text files as the syntax involves the use of more arguments.
- The function `fread()` and `fwrite()` both take 4 arguments to work.

The basic syntax for reading a binary file in C is:

```
fread(data_address, data_size, number_of_data_items, file_pointer);
```

The basic syntax for writing in a binary file in C is:

```
fwrite(data_address, data_size, number_of_data_items, file_pointer);
```

Example 8: Writing a binary file

```
#include <stdio.h>
#include <stdlib.h>

struct threeNum{
int n1, n2, n3;
};

int main()
{
int n;
struct threeNum num;
FILE *fptr;
fptr = fopen("C:\\program.bin","wb"))

if (fptr== NULL){
printf("Error! opening file");
exit(1);
}

for(n = 1; n < 5; ++n)
{
num.n1 = n;
num.n2 = 5*n;
num.n3 = 5*n + 1;
fwrite(&num, sizeof(struct threeNum), 1, fptr);
}
fclose(fptr);
return 0;
}
```

Example 9: Reading a binary file

```
#include <stdio.h>
#include <stdlib.h>

struct threeNum{
int n1, n2, n3;
};

int main()
{
int n;
struct threeNum num;
FILE *fptr;
fptr = fopen("C:\\program.bin","rb"))

if (fptr== NULL){
printf("Error! opening file");
exit(1);
}

for(n = 1; n < 5; ++n)
{
fread(&num, sizeof(struct threeNum), 1, fptr);
printf("n1: %d\\tn2: %d\\tn3: %d\\n", num.n1, num.n2,
num.n3);
}

fclose(fptr);
return 0;

}
```

Example 10: Writing and Reading a binary file

```
#include <stdio.h>

int main()
{
    FILE *fptr = NULL;

    char ch[] = "this is sdf class";

    char str[40];

    fptr = fopen("C:\\\\program.bin","wb+");

    if (fptr== NULL)
    {
        printf("Error! opening file");

        exit(1);
    }
```

```
    fwrite(ch, sizeof(ch), 1, fptr);

    fseek(fptr, 0, SEEK_SET); OR rewind(fptr);

    fread(str, sizeof(str), 1, fptr);

    printf("File content is: %s", str);

    fclose(fptr);

    return 0;
}
```

Output:

File content is: this is sdf class

Three special streams

Three special file streams are defined in the `<stdio.h>` header

- ✓ **stdin** reads input from the keyboard
- ✓ **stdout** send output to the screen
- ✓ **stderr** prints errors to an error device (usually also the screen)

Example: stdin

```
#include <stdio.h>
void main()
{
    char str[25];
    printf("Enter a string:");
    fgets(str, 25, stdin);
    printf("Entered string is %s", str);
}
```

OUTPUT:

Enter a string: Hello World

Entered string is Hello World

Example: stdout

```
#include <stdio.h>
void main()
{
    fprintf(stdout, "Hello World\n");
}
```

OUTPUT:

Hello World

Example: stderr

```
#include <stdio.h>
void main()
{
    int num = 8, num2 = 0;
    if(num2 == 0)
        fprintf(stderr, "Error has occurred\n");
    else
        fprintf(stdout, "%d", num1/num2);
}
```

OUTPUT:

Error has occurred

```
#include <stdio.h>
void main()
{
    int num = 8, num2 = 4;
    if(num2 == 0)
        fprintf(stderr, "Error has occurred\n");
    else
        fprintf(stdout, "%d", num1/num2);
}
```

OUTPUT:

2

Example 11

```
#include <stdio.h>
void main()
{
int i;
fprintf(stdout,"Give value of i \n");
fscanf(stdin,"%d",&i);
fprintf(stdout,"Value of i=%d \n",i);
fprintf(stderr,"No error: But an example to show error message.\n");
}
```

OUTPUT:

Give value of i

15

Value of i=15

No error: But an example to show error message

Command Line Arguments

- We can also give command-line arguments in C
- Command-line arguments are given after the name of the program in command-line shell of Operating Systems.
- To pass command line arguments, we typically define `main()` with two arguments : first argument is the number of command line arguments and second is list of command-line arguments.

int main(int argc, char **argv)

argc (ARGument Count) is int and stores number of command-line arguments passed by the user including the name of the program. So if we pass a value to a program, value of argc would be 2 (one for argument and one for program name)

argv(ARGument Vector) is array of character pointers listing all the arguments.

NOTE:

The value of argc should be non negative.

If argc is greater than zero, the array elements from argv[0] to argv[argc-1] will contain pointers to strings.

Argv[0] is the name of the program , After that till argv[argc-1] every element is command -line arguments.

Properties of Command Line Arguments:

- They are passed to `main()` function.
- They are parameters/arguments supplied to the program when it is invoked.
- They are used to control program from outside instead of hard coding those values inside the code.
- `argv[argc]` is a NULL pointer.
- `argv[0]` holds the name of the program.
- `argv[1]` points to the first command line argument and `argv[n]` points last argument.

Example 12

// Name of program example12.c

```
#include <stdio.h>
```

```
int main(int argc, char *argv[])
{
    int i;
    printf("You have entered %d arguments\n", argc);

    for (i = 0; i < argc; ++i)
        printf("Argument %d = %s\n", i, argv[i]);

    return 0;
}
```

Step 1: Save program by example12.c in D:

Step 2: Compile

Step 3: Run - go to DOS prompt:

D:\ example12 C java python

You have entered 4 arguments

Argument 0 = example12

Argument 1 = C

Argument 2 = java

Argument 3 = python

Example 13

```
#include <stdio.h>
int main(int argc, char *argv[])
{
    while(argc > 1)    /* print out all arguments in reverse order */
    {
        printf("%s\n", argv[argc-1]);
        argc--;
    }
}
```

Save program by example13.c

```
$
$ example13 1 2 3 4
4
3
2
1
```

End of the module